TTLSTT~

TO - TA - LI - SA - TI - ON

Réconciliation avec le produit logiciel

Partie C - Semaines 7-8 et Grande Totalisation

BEASTEST ORIGINALIUM - Méthode d'excellence en qualité logicielle

Livre 2 (L2) - Série BEASTEST

Auteur : Léo Michel Bélanger

Publié par ORIGINALIUM

Longueuil, QC, Canada - 2026

TTLSTT~

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM | TTLSTT~

Mot d'ouverture - Léo Michel Bélanger

Partie C - Semaines 7-8 et Grande Totalisation de L2

Cher·e praticien·ne,

Tu arrives à la dernière ligne droite de ton voyage dans L2 - *Réconciliation avec le produit logiciel*. Ce moment est remarquable. Deux semaines intensives t'ont déjà forgé une compréhension solide de ce qu'est un produit logiciel de qualité, de la façon dont il se construit, se teste et s'évalue. Tu portes maintenant en toi des fondations que peu de professionnels prennent le temps d'établir avec autant de rigueur et d'intention.

La Partie C est la synthèse finale de tout ce travail. Elle ne t'apporte pas simplement deux nouvelles semaines d'exercices - elle te propulse vers une vision complète, cohérente et durable du produit logiciel dans son environnement réel : la production. Tu vas explorer comment la qualité se maintient, se mesure et s'améliore une fois que ton logiciel est entre les mains des utilisateurs. Tu vas également embrasser une perspective holistique et éthique qui fera de toi non seulement un technicien compétent, mais un véritable gardien de l'excellence logicielle.

La Grande Totalisation qui conclut L2 est un moment fort. Elle t'invite à rassembler l'ensemble de tes apprentissages des huit semaines, à les synthétiser en une vision personnelle cohérente, et à te préparer à aborder L3 - le cycle de vie SDLC - avec confiance et clarté.

Bravo pour ton engagement remarquable. Tu l'as mérité, maintenant termine ce que tu as commencé avec tout ce que tu as développé. Allons-y.

Léo Michel Bélanger

Auteur - Série BEASTEST ORIGINALIUM

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM | TTLST~

Table des matières

Section	Contenu
Mot d'ouverture	Lettre de Léo Michel Bélanger
Semaine 7 - Vue d'ensemble	Qualité du produit logiciel en production et feedback loop
Jour 1 - Semaine 7	Monitoring de la qualité en production
Jour 2 - Semaine 7	Gestion des incidents de production
Jour 3 - Semaine 7	Analyse des retours utilisateurs
Jour 4 - Semaine 7	Tests en production - canary, A/B, feature flags
Jour 5 - Semaine 7	Observabilité du produit logiciel
Jour 6 - Semaine 7	Post-mortem et amélioration continue
Jour 7 - Semaine 7	Construire une culture qualité en production
Totalisation - Semaine 7	Exercice de synthèse du feedback loop complet
Semaine 8 - Vue d'ensemble	Vision holistique du produit logiciel et excellence durable
Jour 1 - Semaine 8	Éthique dans le produit logiciel
Jour 2 - Semaine 8	Responsabilité sociale du produit logiciel
Jour 3 - Semaine 8	Innovation et qualité
Jour 4 - Semaine 8	Documentation comme actif de qualité
Jour 5 - Semaine 8	Onboarding qualité
Jour 6 - Semaine 8	Maturité qualité - CMMI, TMMi

Section	Contenu
Jour 7 - Semaine 8	Ton manifeste personnel de qualité
Totalisation - Semaine 8	Exercice de synthèse holistique et éthique
Grande Totalisation - L2	Synthèse complète des 8 semaines
Pont vers L3	Réconciliation avec le cycle de vie SDLC
Pages de notes	3 pages vierges pour tes réflexions

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM | TTLST~

Semaine 7 - Qualité du produit logiciel en production et feedback loop

Objectif de la semaine : Comprendre comment la qualité se vit, se mesure et s'améliore en production - après le déploiement, dans la réalité des utilisateurs.

Intention de la Semaine 7

La production est le véritable terrain de vérité de ton produit logiciel. Ce n'est pas en environnement de test que tu vas découvrir les vraies failles - c'est devant les utilisateurs réels, avec leurs comportements imprévisibles, leurs volumes de données réels et leurs contextes d'usage que tu ne peux pas toujours anticiper. Cette semaine, tu vas outiller ta pensée pour faire de la production un levier d'amélioration continue de la qualité, et non un terrain de

panique.

Vue d'ensemble des thèmes - Semaine 7

Jour	Thème	Compétence développée
Jour 1	Monitoring de la qualité en production	Définir et surveiller les métriques clés de qualité
Jour 2	Gestion des incidents de production	Structurer le processus de réponse aux incidents
Jour 3	Retours utilisateurs comme source de qualité	Analyser et intégrer le feedback utilisateur
Jour 4	Tests en production	Appliquer canary releases, A/B testing et feature flags
Jour 5	Observabilité du produit logiciel	Maîtriser logs, métriques et traces distribuées
Jour 6	Post-mortem et amélioration continue	Apprendre systématiquement des défaillances
Jour 7	Culture qualité en production	Bâtir une organisation apprenante autour de la qualité

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM | TTLST~

Jour 1 - Semaine 7 : Monitoring de la qualité en production - métriques clés, alertes et tableaux de bord

Le monitoring est le système nerveux de ton produit logiciel en production.

Sans lui, tu navigues à l'aveugle. Avec lui, tu as la capacité de détecter, comprendre et répondre aux problèmes de qualité avant qu'ils impactent massivement tes utilisateurs.

Concepts clés

Le monitoring de la qualité en production repose sur trois piliers complémentaires : les **métriques**, les **alertes** et les **tableaux de bord**. Ces trois composantes forment un système de surveillance cohérent qui transforme des données brutes en intelligence opérationnelle.

Les métriques clés de qualité en production

Il existe plusieurs catégories de métriques que tu dois surveiller pour évaluer la qualité de ton produit en production. Les plus importantes sont regroupées selon leur nature :

Catégorie	Métriques	Ce qu'elles révèlent
Disponibilité	Uptime (%), MTBF (Mean Time Between Failures), MTTR (Mean Time To Recovery)	La fiabilité globale du système
Performance	Latence (p50, p95, p99), débit (requêtes/seconde), temps de réponse	L'expérience réelle des utilisateurs
Erreurs	Taux d'erreur (%), codes HTTP 5xx, exceptions non gérées, pannes de services	

Les défaillances fonctionnelles actives

Saturation

Utilisation CPU, mémoire, disque, connexions réseau, files d'attente

L'approche des limites du système

Qualité métier

Transactions complétées, conversions, taux d'abandon, erreurs fonctionnelles

L'impact business des problèmes techniques

Ces quatre premières catégories correspondent au framework **RED** (Rate, Errors, Duration) et **USE** (Utilization, Saturation, Errors) qui sont des approches éprouvées en ingénierie de la fiabilité (SRE).

SLI, SLO et SLA - la hiérarchie de la qualité mesurée

Pour structurer ta démarche de monitoring, tu dois comprendre la hiérarchie des indicateurs de niveau de service :

- **SLI (Service Level Indicator)** : la métrique brute que tu mesures directement, par exemple le taux de requêtes réussies.
- **SLO (Service Level Objective)** : l'objectif interne que ton équipe se fixe, par exemple « 99,9 % de disponibilité sur 30 jours ».
- **SLA (Service Level Agreement)** : l'engagement contractuel envers tes clients, généralement moins exigeant que ton SLO pour absorber les imprévus.
- **Error Budget** : la marge d'erreur acceptable définie par ton SLO - si tu as un SLO de 99,9 %, ton error budget est de 0,1 % d'indisponibilité par période.

Alertes efficaces - éviter la fatigue d'alerte

Un bon système d'alertes est un art délicat. Trop peu d'alertes et tu rates des problèmes critiques. Trop d'alertes et ton équipe développe une *fatigue*

d'alerte (alert fatigue) qui pousse à ignorer même les vraies urgences. Les principes d'une alerte efficace :

- Alerter sur les **symptômes**, pas seulement les causes (ex. : alerte sur l'augmentation de la latence perçue par l'utilisateur plutôt que sur l'utilisation CPU)
- Définir des **seuils adaptés** au contexte - une latence de 500 ms peut être normale pour un rapport complexe et inacceptable pour une page de connexion
- Établir des **niveaux de sévérité** clairs : P1 (critique, intervention immédiate), P2 (urgent, intervention dans l'heure), P3 (important, traitement dans la journée)
- Inclure dans chaque alerte un **runbook** ou un lien vers la procédure de réponse
- Réviser régulièrement tes alertes pour éliminer celles qui sont trop bruyantes ou jamais actionnables

Tableaux de bord - rendre la qualité visible

Un tableau de bord de qualité en production doit permettre à n'importe quel membre de l'équipe de comprendre en 30 secondes si le système est en bonne santé. Les principes de conception :

- Afficher d'abord l'**état global** (vert / jaune / rouge) basé sur le respect des SLO
- Présenter les **métriques des 4 piliers** (disponibilité, performance, erreurs, saturation) de façon visuelle et comparative (maintenant vs. hier, vs. semaine dernière)
- Inclure un **panneau d'incidents actifs** et un historique récent des incidents
- Rendre le tableau de bord **accessible à tous** - développeurs, testeurs,

gestionnaires - chacun doit pouvoir lire l'état de qualité du produit

Exercice - Jour 1

Conception d'un plan de monitoring qualité

Pense à un produit logiciel que tu connais bien (celui sur lequel tu travailles ou un que tu as étudié) et réponds aux questions suivantes :

1. Quelles sont les 5 métriques SLI les plus critiques pour ce produit ? Justifie chacune en termes d'impact sur l'utilisateur.
2. Définis un SLO réaliste pour chacune de ces métriques. Sur quelle base de temps le mesures-tu (24h, 7 jours, 30 jours) ?
3. Pour chaque SLO, quel serait ton error budget mensuel en valeur absolue (minutes d'indisponibilité, nombre d'erreurs autorisées) ?
4. Identifie 3 alertes que tu mettrais en place en priorité. Pour chacune : le seuil, le niveau de sévérité, et l'action attendue.
5. Esquisse la structure d'un tableau de bord en listant les 6 éléments visuels que tu y inclurais, dans l'ordre d'importance.

Questions de réflexion professionnelle

- Est-ce que ton équipe actuelle a des SLO documentés et partagés ? Si non, quels seraient les obstacles à leur mise en place ?
- Comment fais-tu aujourd'hui la différence entre une dégradation de performance normale et un problème de qualité réel ?

- Qui, dans ton organisation, est responsable de répondre aux alertes de production ? Ce rôle est-il clairement défini ?

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM | TTLST~

Jour 2 - Semaine 7 : Gestion des incidents de production - processus, sévérité, résolution et prévention

Un incident de production est un événement non planifié qui impacte la qualité, la disponibilité ou la performance de ton produit logiciel pour tes utilisateurs. La façon dont ton équipe gère ces incidents révèle la maturité réelle de ta culture qualité.

Concepts clés

Le cycle de vie d'un incident

Tout incident de production suit un cycle de vie structuré. Le connaître te permet d'agir avec méthode plutôt que dans la panique :

Phase

Description

Actions clés

Détection

L'incident est identifié, soit par une alerte automatique, soit par un rapport utilisateur

Confirmer l'incident, évaluer l'impact initial, ouvrir le canal de communication

Triage

Évaluation de la sévérité et affectation des ressources

Attribuer une sévérité (P1-P4), assigner un Incident Commander, notifier les parties prenantes

Investigation

Identifier la cause racine ou la cause proximate suffisante pour atténuer

Analyser les logs, métriques, traces ; formuler des hypothèses ; tester des solutions

Mitigation

Réduire ou éliminer l'impact sur les utilisateurs

Rollback, désactivation de feature flag, redirection de trafic, workaround temporaire

Résolution

Le produit retrouve son état nominal

Confirmer le retour à la normale via les métriques, notifier la résolution

Post-mortem

Analyse approfondie pour prévenir la récurrence

Document post-mortem, actions correctives, partage des apprentissages (Jour 6)

Niveaux de sévérité - un langage commun

L'une des erreurs les plus fréquentes est l'absence d'une échelle de sévérité partagée. Sans elle, chaque incident est vécu comme « critique » par celui qui le rapporte, ce qui épuise les équipes et nuit à la qualité des réponses. Voici un modèle standard :

- **P1 - Critique** : Impact total ou majeur sur les utilisateurs, le service est inutilisable. Réponse immédiate requise (minutes). Ex. : site complètement

inaccessible, perte de données en cours.

- **P2 - Majeur** : Impact significatif sur une fonctionnalité clé, un contournement partiel existe. Réponse dans l'heure. Ex. : processus de paiement défaillant, rapports non générés.
- **P3 - Modéré** : Impact sur une fonctionnalité secondaire, les utilisateurs peuvent travailler. Résolution dans la journée. Ex. : un filtre de recherche qui renvoie de mauvais résultats.
- **P4 - Mineur** : Impact cosmétique ou très limité, aucune urgence. Traitement dans le sprint courant. Ex. : un libellé incorrect, une mise en page légèrement décalée.

Le rôle de l'Incident Commander

Pour les incidents P1 et P2, nommer un **Incident Commander** est une pratique essentielle. Ce rôle ne répare pas le problème - il orchestre la réponse : qui fait quoi, qui communique vers l'externe, qui documente en temps réel. Séparer le rôle de coordination du rôle de résolution technique est l'une des décisions organisationnelles qui fait le plus de différence dans la gestion efficace des incidents.

Communication pendant un incident - la transparence comme actif qualité

La façon dont tu communiques pendant un incident affecte directement la perception de la qualité de ton produit par tes utilisateurs et tes parties prenantes :

- Mettre à jour ta **page de statut** dans les 5-10 minutes suivant la détection d'un P1
- Communiquer à intervalles réguliers (toutes les 30 minutes pour un P1), même si tu n'as pas de nouvelle information - « Nous travaillons toujours sur le problème » rassure plus que le silence

- Être précis sur l'impact réel, sans sur-dramatiser ni sous-estimer
- Annoncer la résolution avec une explication succincte de la cause

Prévention - apprendre avant que ça arrive

La meilleure gestion d'incident est celle qui n'a jamais lieu. Les pratiques préventives incluent :

- **Chaos engineering** : introduire intentionnellement des défaillances en environnement contrôlé pour tester la résilience du système
- **Game days** : simuler des scénarios d'incidents pour entraîner l'équipe et valider les procédures
- **Capacity planning** : anticiper la croissance du trafic et s'assurer que l'infrastructure peut absorber les pics
- **Review des changements** : la majorité des incidents P1 sont causés par des changements récents - un processus de validation rigoureux des déploiements est ta première ligne de défense

Exercice - Jour 2

Conception d'un playbook d'incident

1. Crée une matrice de sévérité adaptée à ton contexte professionnel avec des exemples concrets pour chaque niveau (P1 à P4) tirés de ton domaine applicatif.
2. Rédige un runbook simplifié pour un scénario d'incident P1 typique dans ton domaine (ex. : base de données inaccessible, service d'authentification en panne). Inclus les étapes de détection, triage, mitigation et communication.
3. Identifie les 3 types d'incidents les plus fréquents dans ton contexte

actuel. Pour chacun, quelle est la cause racine la plus probable ? Quelle action préventive pourrait réduire sa probabilité de 50 % ?

4. Évalue la maturité de ton processus de gestion d'incidents actuel sur une échelle de 1 à 5 pour chaque dimension : (a) détection, (b) communication, (c) résolution, (d) prévention. Justifie chaque note.

Questions de réflexion professionnelle

- As-tu déjà vécu un incident de production mal géré ? Qu'est-ce qui a manqué dans le processus de réponse ?
- Dans ton équipe, qui est responsable de l'escalade lors d'un incident critique ? Ce rôle est-il documenté et connu de tous ?
- Comment mesures-tu le MTTR (Mean Time To Recovery) de ton produit ? As-tu un objectif défini ?

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM | TTLST~

Jour 3 - Semaine 7 : Analyse des retours utilisateurs comme source de qualité - le feedback loop

Les utilisateurs sont tes testeurs les plus précieux - et les plus impitoyables. Chaque retour qu'ils formulent, qu'il soit positif, négatif ou sous la forme d'un comportement d'usage, contient une information de qualité que tes tests automatisés n'ont pas pu capturer. Apprendre à collecter, analyser et intégrer ce feedback dans ta démarche qualité est une compétence fondamentale.

Concepts clés

Les sources de feedback utilisateur

Le feedback utilisateur arrive par de nombreux canaux, et chacun révèle une dimension différente de la qualité perçue :

Source	Type de signal	Indicateur qualité associé
Tickets de support	Explicite, structuré	Problèmes fonctionnels, bugs, confusion UX
Sondages in-app (NPS, CSAT)	Explicite, quantifié	Satisfaction générale, perception de la qualité
Commentaires et avis (stores, forums)	Explicite, non structuré	Points de friction majeurs, fonctionnalités manquantes
Sessions d'utilisation (analytics)	Implicite, comportemental	Taux d'abandon, parcours atypiques, erreurs répétées
Enregistrements de session	Implicite, visuel	Friction UX, rage clicks, confusion de navigation

Entretiens utilisateurs

Explicite, qualitatif

Contexte d'usage, besoins non satisfaits, modèle mental

Le feedback loop - la boucle de rétroaction qualité

Un feedback loop efficace transforme les retours utilisateurs en améliorations concrètes du produit. Il comprend cinq étapes :

1. **Collecte** : centraliser tous les signaux dans un système unifié (ex. : un outil de ticketing qui agrège support, analytics et sondages)
2. **Catégorisation** : tagger chaque retour par type (bug, demande de fonctionnalité, problème de performance, confusion UX) et par composante du produit affectée
3. **Priorisation** : évaluer l'impact (combien d'utilisateurs affectés, quelle fréquence, quelle sévérité de l'impact) et l'effort de résolution
4. **Action** : transformer les retours prioritaires en tâches concrètes dans ton backlog avec un lien traçable vers le retour original
5. **Clôture** : informer les utilisateurs (quand possible) des améliorations effectuées suite à leurs retours - cette étape renforce la confiance et encourage de futurs retours de qualité

Le NPS (Net Promoter Score) comme indicateur de qualité perçue

Le NPS mesure la probabilité qu'un utilisateur recommande ton produit à un collègue ou un ami, sur une échelle de 0 à 10. Les utilisateurs sont classés en **Promoteurs** (9-10), **Passifs** (7-8) et **Détracteurs** (0-6). Le score NPS est calculé par la formule : % Promoteurs - % Détracteurs.

L'outil le plus précieux n'est pas le score lui-même, mais la question ouverte qui l'accompagne : « Qu'est-ce qui vous a amené à donner cette note ? » Les verbatims des détracteurs sont une mine d'or de problèmes de qualité à

traiter.

Transformer les retours négatifs en force qualité

La réaction naturelle face à un retour négatif est défensive. La réaction mature, celle d'un praticien qualité aguerri, est analytique :

- Chaque bug rapporté par un utilisateur est une lacune dans ta stratégie de test - qu'est-ce qui t'a empêché de le détecter avant ?
- Chaque plainte de confusion UX est un signal que ton produit ne communique pas clairement sa valeur - comment peux-tu améliorer la clarté ?
- Chaque demande de fonctionnalité répétée révèle un besoin réel non satisfait - est-ce dans ton scope ? Si oui, quelle priorité ?

Exercice - Jour 3

Conception d'un système de feedback loop qualité

1. Cartographie les sources de feedback utilisateur disponibles dans ton contexte actuel. Lesquelles sont utilisées activement ? Lesquelles sont ignorées ? Pourquoi ?
2. Choisis 10 retours utilisateurs réels (ou simulés à partir de ton expérience) et applique le processus de catégorisation : type, composante affectée, sévérité, fréquence estimée.
3. Parmi ces 10 retours, priorise-les selon une matrice impact/effort et identifie les 3 à traiter en priorité. Rédige pour chacun une tâche backlog concrète et traçable.
4. Conçois une question NPS et sa question ouverte d'accompagnement adaptée à ton produit logiciel. Comment intégrerait-il les verbatims dans ton

processus qualité ?

Questions de réflexion professionnelle

- Dans ton organisation, qui est responsable d'analyser les retours utilisateurs ? Est-ce une responsabilité partagée avec l'équipe qualité ?
- As-tu déjà identifié un problème de qualité grâce aux retours utilisateurs que tes tests n'avaient pas détecté ? Qu'as-tu appris de cette expérience ?
- Comment mesures-tu la satisfaction de tes utilisateurs aujourd'hui ? Le score obtenu est-il utilisé pour piloter des décisions de qualité concrètes ?

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM | ŢŢĻŞŢ~

Jour 4 - Semaine 7 : Tests en production - canary releases, A/B testing, feature flags et qualité

L'idée de « tester en production » peut paraître risquée à première vue. En réalité, les techniques modernes de déploiement progressif te permettent de valider la qualité de ton produit directement avec des utilisateurs réels, tout en maîtrisant le risque. C'est l'une des avancées les plus significatives de l'ingénierie logicielle des dix dernières années.

Concepts clés

Canary Releases - déploiement progressif et qualité contrôlée

Un canary release consiste à déployer une nouvelle version de ton produit à un sous-ensemble limité d'utilisateurs (typiquement 1 % à 10 %) avant de

l'étendre à l'ensemble de la base. Le nom vient de la pratique historique des mineurs qui emmenaient un canari dans les mines pour détecter les gaz toxiques avant qu'ils n'atteignent des concentrations dangereuses pour les humains.

Le processus de canary release pour la qualité :

1. Déployer la nouvelle version à un petit groupe d'utilisateurs pilotes (ou à un pourcentage aléatoire)
2. Monitorer intensivement les métriques de qualité (taux d'erreur, latence, comportement) pendant une période définie (quelques heures à quelques jours)
3. Comparer statistiquement les métriques du groupe canary vs. le groupe de contrôle
4. Si les métriques sont satisfaisantes, augmenter progressivement le pourcentage (1 % → 5 % → 25 % → 100 %)
5. Si une dégradation est détectée, effectuer un rollback immédiat du groupe canary sans impact sur le reste des utilisateurs

A/B Testing - valider la qualité fonctionnelle et l'impact utilisateur

L'A/B testing te permet de comparer deux versions d'une fonctionnalité pour déterminer laquelle produit les meilleurs résultats selon des métriques définies. Dans une perspective qualité :

Dimension

Version A (contrôle)

Version B (variante)

Métrique de qualité

Performance perçue

Chargement standard	
Chargement optimisé avec lazy loading	
LCP (Largest Contentful Paint), taux d'abandon	
Fiabilité fonctionnelle	
Formulaire sans validation temps réel	
Formulaire avec validation inline	
Taux d'erreur de soumission, tickets de support	
Accessibilité	
Navigation par clic standard	
Navigation améliorée au clavier	
Taux de complétion pour les utilisateurs assistifs	

Feature Flags - contrôle granulaire de la qualité

Les feature flags (ou feature toggles) sont des interrupteurs configurables qui permettent d'activer ou désactiver une fonctionnalité sans nouveau déploiement. Leurs applications en qualité sont nombreuses :

- **Kill switch** : désactiver immédiatement une fonctionnalité défaillante sans rollback complet du déploiement
- **Déploiement découplé du lancement** : déployer du code en production mais ne l'activer qu'au moment choisi, après validation complète
- **Tests bêta ciblés** : activer une fonctionnalité uniquement pour un groupe d'utilisateurs bêta ou internes avant le lancement général
- **Dégradation gracieuse** : en cas de problème sur un service tiers, désactiver la fonctionnalité dépendante plutôt que de laisser une erreur impacter l'expérience
- **Test de charge progressif** : activer une nouvelle architecture pour un

pourcentage croissant d'utilisateurs pour tester sa résistance sous charge réelle

Les risques et garde-fous

Tester en production n'est pas une licence pour abandonner les tests préalables. Les garde-fous essentiels :

- Maintenir une suite de tests robuste en amont (unitaires, intégration, E2E) - les tests en production complètent, ils ne remplacent pas
- Définir des critères d'arrêt automatique (automatic rollback triggers) basés sur les métriques
- S'assurer que les données des utilisateurs du groupe canary sont protégées et que les erreurs éventuelles n'ont pas de conséquences irréversibles
- Documenter et communiquer clairement à l'équipe quels feature flags sont actifs et pour quels utilisateurs

Exercice - Jour 4

Planification d'un déploiement canary et d'un A/B test qualité

1. Identifie une fonctionnalité récente ou à venir dans ton contexte. Conçois un plan de canary release pour elle : population initiale, métriques surveillées, critères de progression, critères de rollback.
2. Pour la même fonctionnalité ou une autre, conçois un A/B test axé sur la qualité. Définis : la variable testée, la métrique primaire, la métrique secondaire, la durée minimale du test, et les critères de décision.
3. Identifie 3 fonctionnalités de ton produit qui bénéficieraient d'un feature flag. Pour chacune, explique quel risque le feature flag permettrait de

mitiger.

4. Évalue les conditions préalables manquantes dans ton organisation pour déployer des canary releases aujourd'hui. Quels sont les 2 obstacles principaux et comment les adresserais-tu ?

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM | TTLST~

Jour 5 - Semaine 7 : Observabilité du produit logiciel - logs, métriques, traces distribuées

L'observabilité est la capacité de comprendre l'état interne de ton système à partir de ses sorties externes. Contrairement au monitoring, qui te dit quand quelque chose ne va pas, l'observabilité te permet de comprendre *pourquoi* quelque chose ne va pas - même pour des problèmes que tu n'avais pas anticipés.

Concepts clés

Les trois piliers de l'observabilité

Pilier

Définition

Ce qu'il révèle

Outils courants

Logs

Enregistrements textuels horodatés des événements du système

Quoi s'est passé, quand, dans quel contexte

Elasticsearch, Splunk, Loki, CloudWatch

Métriques

Données numériques agrégées et horodatées sur le comportement du système

Tendances, anomalies, dégradation dans le temps

Prometheus, Grafana, Datadog, New Relic

Traces

Enregistrement du chemin complet d'une requête à travers les services

Où la latence est introduite, quel service est défaillant

Jaeger, Zipkin, AWS X-Ray, OpenTelemetry

Logs - les journaux de ton produit logiciel

Un log de qualité contient cinq éléments essentiels :

- **Timestamp précis** : au moins à la milliseconde, en UTC pour éviter les problèmes de fuseau horaire
- **Niveau de sévérité** : DEBUG, INFO, WARN, ERROR, FATAL - utilisés de façon cohérente
- **Message descriptif** : clair, actionnable, sans données personnelles en clair
- **Contexte structuré** : ID de requête, ID utilisateur anonymisé, composante, environnement
- **Stack trace** : pour les erreurs, le contexte complet de l'exception

La pratique du **structured logging** (JSON ou format structuré) plutôt que du logging texte libre est essentielle pour permettre l'analyse automatisée et les recherches efficaces à grande échelle.

Métriques - quantifier la qualité dans le temps

Les métriques d'observabilité se distinguent des métriques de monitoring par

leur granularité et leur utilisabilité dans le diagnostic. Les types de métriques d'observabilité :

- **Counters** : compteurs qui ne font qu'augmenter (nombre de requêtes, nombre d'erreurs)
- **Gauges** : valeurs qui peuvent monter et descendre (utilisation mémoire, connexions actives)
- **Histogrammes** : distribution de valeurs sur des buckets (distribution des temps de réponse)
- **Percentiles** : p50, p95, p99 - essentiels pour comprendre l'expérience réelle, car les moyennes masquent les cas extrêmes

Traces distribuées - suivre le chemin complet

Dans une architecture à services multiples (microservices), une requête utilisateur peut traverser 5, 10, voire 20 services différents avant de retourner une réponse. Les traces distribuées te permettent de suivre ce chemin complet avec une granularité temporelle précise.

Chaque trace est composée de **spans** - des unités de travail avec un début, une fin et des métadonnées. L'analyse d'une trace te révèle :

- Quel service introduit le plus de latence
- Quelles dépendances sont appelées séquentiellement (et pourraient être parallélisées)
- Où les erreurs se propagent dans la chaîne
- Quelles requêtes DB, appels API externes ou opérations I/O sont les plus coûteuses

OpenTelemetry - le standard d'observabilité

OpenTelemetry (OTel) est devenu le standard ouvert de l'industrie pour l'instrumentation de l'observabilité. Il fournit un SDK unifié pour collecter logs, métriques et traces, et les exporter vers n'importe quel backend

d'observabilité. Adopter OpenTelemetry t'offre la flexibilité de changer d'outil sans réinstrumenter ton code.

Exercice - Jour 5

Audit d'observabilité de ton produit logiciel

1. Évalue l'état actuel de tes logs : sont-ils structurés ? Contiennent-ils les 5 éléments essentiels ? Peuvent-ils être recherchés efficacement ? Identifie les 3 améliorations prioritaires.
2. Identifie les 5 métriques les plus importantes que ton produit devrait exposer mais ne mesure pas encore. Pour chacune, spécifie le type (counter, gauge, histogramme), la fréquence de collecte et l'outil de collecte recommandé.
3. Dessine (ou décris textuellement) le chemin d'une requête critique de ton produit à travers tes services. Combien de services traversée ? Quels sont les points de défaillance potentiels ? Où la latence est-elle la plus difficile à comprendre actuellement ?
4. Recherche les capacités d'observabilité de ta plateforme de déploiement actuelle (cloud, on-premise). Quelles fonctionnalités natives d'observabilité sont disponibles mais non exploitées ?

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM | TTLST~

Jour 6 - Semaine 7 : Post-mortem et amélioration continue - tirer les leçons

des défaillances

Un post-mortem est bien plus qu'un rapport d'incident. C'est un rituel organisationnel de transformation : celui qui convertit la douleur d'une défaillance en sagesse collective. Les équipes qui maîtrisent l'art du post-mortem sans reproches (blameless post-mortem) évoluent plus rapidement et construisent des produits logiciels de qualité supérieure.

Concepts clés

Le post-mortem sans reproches - philosophie fondamentale

La philosophie du **blameless post-mortem**, popularisée par Google SRE et des organisations comme Etsy, repose sur un principe fondamental : les personnes qui travaillent dans des systèmes complexes font des erreurs. Ces erreurs sont des symptômes de problèmes systémiques, pas des preuves d'incompétence individuelle.

Blâmer une personne pour un incident :

- Décourage le signalement futur des problèmes (les gens cachent les erreurs pour se protéger)
- S'attaque au symptôme (l'action humaine) plutôt qu'à la cause racine (le système défaillant)
- Crée une culture de peur qui est incompatible avec l'excellence en qualité

La question n'est jamais « qui a causé le problème ? », mais toujours « comment notre système a-t-il permis que ce problème arrive et se propage ? »

Structure d'un post-mortem efficace

Section

Contenu

Objectif

Résumé de l'incident

Date, durée, services affectés, impact utilisateur (utilisateurs impactés, SLO violé, SLA impacté)

Contexte partagé pour tous les lecteurs

Chronologie

Timeline horodatée de la détection à la résolution, incluant toutes les actions et décisions

Comprendre la séquence exacte des événements

Analyse des causes racines

Technique des 5 Pourquoi appliquée pour identifier la cause profonde (pas juste la cause directe)

Trouver le levier systémique de changement

Ce qui a bien fonctionné

Les pratiques et mécanismes qui ont limité l'impact ou accéléré la résolution

Renforcer les comportements positifs

Ce qui aurait pu mieux fonctionner

Les lacunes dans la détection, la réponse ou la communication

Identifier les améliorations nécessaires

Actions correctives

Liste des actions concrètes avec propriétaire et délai définis

Garantir que les apprentissages se traduisent en améliorations réelles

La technique des 5 Pourquoi

La technique des 5 Pourquoi, développée par Toyota dans le cadre du

Système de Production Toyota, est l'outil d'analyse de cause racine le plus accessible et le plus puissant pour les équipes logicielles. Exemple simplifié :

- **Pourquoi 1** : Pourquoi le service de paiement était-il inaccessible ? - Parce que la base de données a atteint sa limite de connexions.
- **Pourquoi 2** : Pourquoi la base de données a-t-elle atteint sa limite de connexions ? - Parce qu'un processus de batch en arrière-plan ouvrait des connexions sans les fermer.
- **Pourquoi 3** : Pourquoi ce processus ouvrait-il des connexions sans les fermer ? - Parce qu'une exception non gérée interrompait le processus avant la fermeture.
- **Pourquoi 4** : Pourquoi l'exception n'était-elle pas gérée ? - Parce que ce cas d'erreur n'avait pas été couvert dans les tests.
- **Pourquoi 5** : Pourquoi ce cas d'erreur n'avait-il pas été couvert ? - Parce qu'il n'existait pas de pratique de revue de couverture de test pour les processus batch.

La cause racine systémique est donc : l'absence d'un processus de revue de couverture de test pour les processus batch. L'action corrective est de créer ce processus, pas de « réparer le bug » seulement.

Fermer la boucle - de l'apprentissage à l'action

Un post-mortem sans actions correctives suivies est un exercice de style. Pour garantir que les apprentissages se traduisent en améliorations :

- Chaque action corrective doit avoir un **propriétaire unique** (pas une équipe, une personne)
- Chaque action doit avoir une **date d'échéance** réaliste et suivie en revue d'équipe
- Les post-mortems doivent être **partagés** au-delà de l'équipe affectée - le reste de l'organisation peut apprendre des mêmes leçons

- Mesurer le **taux de complétion** des actions correctives comme indicateur de maturité qualité

Exercice - Jour 6

Rédiger ton premier post-mortem blameless

1. Choisis un incident de production réel (ou reconstitue-en un hypothétique crédible à partir de ton expérience). Rédige un post-mortem complet selon la structure présentée.
2. Applique la technique des 5 Pourquoi à la cause identifiée. Jusqu'à quelle profondeur arrives-tu ? Où la chaîne de causalité t'amène-t-elle ?
3. Identifie 3 actions correctives concrètes avec propriétaire et délai. Comment t'assureras-tu de leur complétion ?
4. Réfléchis à la culture de ton équipe actuelle : une discussion post-mortem blameless serait-elle possible ? Quels obstacles culturels ou organisationnels existent ? Propose une stratégie pour les surmonter.

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM | TTLST~

Jour 7 - Semaine 7 : Construire une culture qualité autour du produit logiciel en production

La culture qualité est la dimension la plus difficile à construire et la plus difficile à détruire une fois bien établie. Elle transcende les outils, les

processus et les méthodologies : c'est l'ensemble des valeurs, comportements et pratiques partagés qui font que chaque membre de l'équipe se sent responsable de la qualité du produit en production.

Concepts clés

Les marqueurs d'une culture qualité mature

Dimension

Culture qualité immature

Culture qualité mature

Responsabilité

« C'est la faute du QA de ne pas avoir trouvé le bug »

« La qualité est la responsabilité de toute l'équipe »

Réponse aux incidents

Panique, blâme, silotage de l'information

Calme, collaboration, transparence, apprentissage

Relation avec la production

« La production est l'affaire des ops »

« Tu construis, tu l'opères » (You build it, you run it)

Rapport aux erreurs

Minimiser, cacher, éviter le sujet

Signaler, analyser, partager les apprentissages

Mesure de la qualité

Subjective (« ça semble fonctionner »)

Objective (métriques, SLO, dashboards partagés)

Innovation vs. stabilité

Tension permanente, opposition binaire

Équilibre géré par error budget et déploiement progressif

Le principe « You build it, you run it »

Ce principe, popularisé par Amazon, stipule que les équipes qui développent un service sont également responsables de son opération en production. Cette approche a un impact transformateur sur la qualité :

- Les développeurs conçoivent avec l'opérabilité en tête (observabilité, gestion des erreurs, dégradation gracieuse)
- Ils sont directement exposés aux conséquences de leurs décisions de conception
- Le feedback entre production et développement est immédiat et direct
- La frontière artificielle entre « développement » et « opérations » disparaît

L'on-call comme levier culturel de qualité

La gestion de la permanence (on-call) est un sujet sensible, mais bien structurée, elle est l'un des leviers les plus puissants de culture qualité. Les pratiques d'une on-call saine :

- Rotation équitable avec des plages clairement définies et respectées
- Outillage de qualité pour répondre aux incidents efficacement (runbooks, tableaux de bord, accès aux outils)
- Compensation claire pour les interruptions hors heures ouvrables
- Revue régulière de la charge on-call pour garantir qu'elle reste raisonnable
- Un contrat implicite : si l'on-call est trop exigeante, c'est le signal que la

qualité du produit en production doit être améliorée en priorité

Les rituels de qualité en production

Les rituels sont les pratiques récurrentes qui incarnent la culture. Les rituels de qualité en production les plus efficaces :

- **Weekly SLO review** : revue hebdomadaire des SLO avec toute l'équipe, incluant la consommation d'error budget
- **Incident retrospective** : partage régulier des post-mortems en format « lunch and learn » ouvert à d'autres équipes
- **Production readiness review** : évaluation formelle avant tout déploiement majeur (observabilité configurée, alertes en place, runbooks prêts, plan de rollback testé)
- **On-call handoff** : passation structurée entre les personnes de garde avec un résumé de l'état actuel du système

Exercice - Jour 7

Diagnostiquer et renforcer ta culture qualité en production

1. Positionne ton équipe actuelle sur chaque dimension du tableau « culture immature vs. mature ». Donne une note de 1 à 5 pour chaque dimension et justifie.
2. Identifie le rituel de qualité en production qui aurait le plus grand impact dans ton contexte actuel. Rédige un plan d'implémentation sur 90 jours pour l'introduire.
3. Comment le principe « You build it, you run it » pourrait-il transformer la dynamique de ton équipe ? Quels bénéfices anticipes-tu ? Quels obstacles vois-tu ?

4. Rédige une « Production Readiness Checklist » de 10 à 15 items adaptée à ton contexte, que ton équipe pourrait utiliser avant chaque déploiement majeur.

Questions de réflexion professionnelle - Clôture Semaine 7

- Si un nouvel ingénieur rejoignait ton équipe demain, quelle image de la culture qualité en production percevrait-il dans les deux premières semaines ?
- Quel est le rituel de qualité que tu introduirais en premier si tu avais le mandat de ton gestionnaire pour le faire ?
- Comment la culture qualité de ton organisation a-t-elle évolué depuis les 12 derniers mois ? Dans quelle direction ?

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM | TTLST~

TTLST~ - Totalisation - Semaine 7

Qualité du produit logiciel en production et feedback loop

Les deux dimensions de la Totalisation

Dimension 1 - Substantif linguistique

"Action d'additionner ; action de considérer l'ensemble, le total, de synthétiser."

Dimension 2 - Philosophie ORIGINALIUM

"Rassembler et réaliser totalement dans son ensemble - avant, pendant et après - en continu, sans interruption."

Exercice de Totalisation - Semaine 7

Totaliser la Semaine 7, c'est comprendre que la production n'est pas la fin du voyage de la qualité - c'est là où la qualité *vit vraiment*. Tu vas maintenant synthétiser le feedback loop complet du produit logiciel.

Synthèse des 7 jours

Jour	Apprentissage essentiel	Connexion au feedback loop
1 - Monitoring	Mesurer en continu pour détecter avant l'utilisateur	Point de départ : signal objectif de la réalité
2 - Incidents	Répondre avec méthode, communiquer avec transparence	Validation de la robustesse du feedback loop en situation de crise
3 - Retours utilisateurs	Le feedback humain révèle ce que les métriques ne voient pas	Enrichissement qualitatif du signal de qualité
4 - Tests en production	Valider progressivement pour maîtriser le risque	Fermeture de la boucle : action contrôlée, mesure, décision
5 - Observabilité	Comprendre l'état interne par les sorties externes	Infrastructure cognitive du feedback loop

Jour	Apprentissage essentiel	Connexion au feedback loop
6 - Post-mortem	Apprendre systématiquement de chaque défaillance	Mécanisme d'amélioration continue du feedback loop lui-même
7 - Culture qualité	Les pratiques ne durent que si la culture les soutient	Terreau dans lequel le feedback loop peut croître et prospérer

Exercice de Totalisation - Semaine 7

Synthétise le feedback loop complet de ton produit logiciel en production en répondant à ces questions intégratrices :

1. **Le signal** : Quels sont les 3 signaux les plus importants que tu surveilles (ou devrais surveiller) pour évaluer la qualité de ton produit en production ? Comment sont-ils collectés, stockés et rendus visibles ?
2. **L'analyse** : Quand un signal de qualité dégradée est détecté, quel est le processus complet de ton équipe pour l'analyser et comprendre sa cause ? Où sont les lacunes ?
3. **L'action** : Comment les apprentissages de la production se traduisent-ils concrètement en améliorations du produit ? Ce chemin est-il traçable ?
4. **La boucle** : Dessine ou décris textuellement le feedback loop complet de ton produit logiciel - de la détection d'un problème en production à son amélioration vérifiée. Combien de temps prend cette boucle complète aujourd'hui ? Quel serait l'idéal ?
5. **La totalisation personnelle** : En une ou deux phrases, formule ta compréhension synthétisée de la relation entre la production et la qualité du produit logiciel, basée sur tout ce que tu as appris cette semaine.

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM | TTLST~

Semaine 8 - Vision holistique du produit logiciel et excellence durable

Objectif de la semaine : Embrasser une vision complète, éthique et humaine du produit logiciel de qualité - au-delà de la technique, vers l'excellence durable.

Intention de la Semaine 8

La Semaine 8 est la semaine de la vision élargie. Tu as construit une compréhension solide du produit logiciel dans ses dimensions techniques, fonctionnelles et opérationnelles. Il est maintenant temps d'élargir ta perspective vers les dimensions éthiques, humaines et organisationnelles qui conditionnent l'excellence durable. Un produit logiciel de qualité n'est pas seulement correct, rapide et fiable - il est aussi juste, responsable, innovant et humain.

Vue d'ensemble des thèmes - Semaine 8

Jour	Thème	Compétence développée
Jour 1	Éthique dans le produit logiciel	Intégrer accessibilité, confidentialité et sécurité comme piliers qualité
Jour 2	Responsabilité sociale	Évaluer l'impact humain et sociétal du produit

Jour	Thème	Compétence développée
		logiciel
Jour 3	Innovation et qualité	Innover sans sacrifier la fiabilité
Jour 4	Documentation comme actif qualité	Structurer wikis, ADR et runbooks
Jour 5	Onboarding qualité	Intégrer la qualité dès l'arrivée de nouveaux membres
Jour 6	Maturité qualité - CMMI, TMMi	Évaluer et piloter la progression en maturité qualité
Jour 7	Manifeste personnel de qualité	Articuler et formaliser ta vision personnelle de la qualité

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM | TTLST~

Jour 1 - Semaine 8 : L'éthique dans le produit logiciel - accessibilité, confidentialité, sécurité comme piliers de qualité

L'éthique dans le produit logiciel n'est pas un sujet réservé aux philosophes ou aux juristes. C'est un ensemble de dimensions concrètes qui déterminent si ton produit respecte vraiment les personnes qui l'utilisent. Accessibilité, confidentialité et sécurité ne sont pas des fonctionnalités optionnelles - ce sont des piliers fondamentaux de la qualité.

Concepts clés

Accessibilité - la qualité pour tous les utilisateurs

Un produit logiciel accessible est un produit qui peut être utilisé par des personnes ayant des capacités diverses : troubles visuels, auditifs, moteurs,

cognitifs. Les standards WCAG (Web Content Accessibility Guidelines) définissent les critères d'accessibilité selon quatre principes fondamentaux :

Principe WCAG

Description

Exemples concrets

Perceptible

L'information doit être présentable aux utilisateurs de façon qu'ils puissent la percevoir

Texte alternatif pour les images, sous-titres pour les vidéos, contraste de couleur suffisant

Utilisable

Les composants d'interface et de navigation doivent être utilisables

Navigation complète au clavier, temps suffisant pour lire, pas de contenu qui provoque des crises

Compréhensible

L'information et l'interface doivent être compréhensibles

Langue du contenu déclarée, navigation cohérente, messages d'erreur descriptifs

Robuste

Le contenu doit être assez robuste pour être interprété par les technologies d'assistance

HTML valide, attributs ARIA corrects, compatibilité avec les lecteurs d'écran

L'accessibilité est une obligation légale dans de nombreuses juridictions (ADA aux États-Unis, Loi sur l'accessibilité pour les personnes handicapées de l'Ontario, RQAP au Québec) et un impératif éthique universel. Tester

l'accessibilité doit faire partie de ta stratégie de test, au même titre que les tests fonctionnels.

Confidentialité - la qualité qui protège les personnes

La confidentialité (privacy) est la dimension qualité qui garantit que les données personnelles de tes utilisateurs sont collectées, traitées et protégées de façon éthique et légale. Les principes fondamentaux du Privacy by Design :

- **Minimisation des données** : ne collecter que ce qui est strictement nécessaire
- **Limitation de la finalité** : n'utiliser les données que pour l'usage déclaré
- **Consentement éclairé** : obtenir un consentement clair, spécifique et révocable
- **Droit à l'oubli** : permettre aux utilisateurs de supprimer leurs données
- **Transparence** : documenter et communiquer clairement l'usage des données
- **Sécurité par défaut** : les paramètres par défaut doivent être les plus protecteurs possible

Sécurité comme pilier de qualité - pas un ajout, une fondation

La sécurité est souvent traitée comme une préoccupation distincte de la qualité. C'est une erreur conceptuelle. Un produit logiciel non sécurisé n'est pas un produit de qualité, peu importe ses performances ou sa fiabilité fonctionnelle. Les pratiques de sécurité qui appartiennent à la qualité :

- **Threat modeling** : identifier et évaluer les menaces potentielles dès la conception
- **SAST (Static Application Security Testing)** : analyse du code source pour détecter les vulnérabilités
- **DAST (Dynamic Application Security Testing)** : tests de sécurité sur

l'application en cours d'exécution

- **Dependency scanning** : surveillance continue des vulnérabilités dans les bibliothèques tierces
- **Penetration testing** : tests d'intrusion réguliers pour valider la robustesse des défenses
- **OWASP Top 10** : connaître et traiter les 10 risques de sécurité les plus critiques des applications web

Exercice - Jour 1

Audit éthique de ton produit logiciel

1. Évalue le niveau d'accessibilité actuel de ton produit sur les 4 principes WCAG. Pour chaque principe, identifie un point fort et une lacune concrète.
2. Cartographie les données personnelles collectées par ton produit : quelles données, pour quelle finalité, avec quel consentement, quelle durée de rétention. Y a-t-il des données collectées sans nécessité claire ?
3. Quels tests de sécurité sont actuellement intégrés à ton pipeline CI/CD ? Lesquels manquent ? Classe-les par priorité d'implémentation.
4. Identifie un aspect éthique (accessibilité, confidentialité ou sécurité) qui représente le risque qualité le plus élevé dans ton contexte. Rédige un plan d'action sur 60 jours pour l'adresser.

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM | TTLST~

Jour 2 - Semaine 8 : Produit logiciel et responsabilité sociale - qualité dans un contexte humain et sociétal

Chaque produit logiciel existe dans un contexte humain et sociétal. Il est utilisé par des personnes réelles, dans des contextes réels, avec des impacts réels sur leur vie, leur travail et leur bien-être. La responsabilité sociale du produit logiciel est la reconnaissance de cette réalité et l'engagement à en tenir compte dans chaque décision de qualité.

Concepts clés

L'impact sociétal du produit logiciel de qualité

Un produit logiciel mal conçu peut avoir des conséquences bien au-delà de la frustration utilisateur :

- Un logiciel médical défaillant peut mettre des vies en danger
- Un algorithme de crédit biaisé peut perpétuer des inégalités systémiques
- Un système de recrutement automatisé avec des biais peut discriminer des candidats qualifiés
- Une application de communication non sécurisée peut exposer des journalistes ou des militants à des risques réels
- Un produit logiciel conçu pour créer de la dépendance (dark patterns) exploite ses utilisateurs

La qualité, dans ce contexte, c'est aussi la capacité de ton produit à *ne pas causer de tort* - principe fondamental de l'éthique médicale (primum non nocere) que l'ingénierie logicielle doit adopter.

Les biais dans les produits logiciels

Les biais peuvent s'introduire dans ton produit à plusieurs niveaux :

Type de biais

Origine

Impact sur la qualité

Biais dans les données d'entraînement

Données historiques reflétant des inégalités passées

Algorithmes reproduisant et amplifiant des discriminations

Biais de conception

Équipes de conception non représentatives

Produit inadapté à certains groupes d'utilisateurs

Biais de test

Tests réalisés uniquement sur des cas majoritaires

Problèmes invisibles pour les utilisateurs hors norme

Dark patterns

Conception intentionnellement trompeuse

Manipulation de l'utilisateur contre ses propres intérêts

La qualité inclusive - concevoir et tester pour la diversité

L'inclusion comme pratique qualité signifie élargir délibérément le spectre des utilisateurs représentés dans ta démarche qualité :

- Inclure des personas diversifiés (âge, culture, niveau technique, capacités) dans ta stratégie de test
- Réaliser des tests d'utilisabilité avec des participants représentatifs de la diversité de ta base d'utilisateurs
- Évaluer les cas aux limites et les scénarios hors norme avec autant de rigueur que les cas principaux
- Intégrer des perspectives diverses dans tes équipes de test et de

conception

La durabilité numérique comme dimension qualité

La durabilité numérique (Green IT) est une dimension de responsabilité sociale émergente mais incontournable. L'empreinte environnementale de ton produit logiciel - consommation énergétique des serveurs, des transferts de données, des appareils clients - est une composante de sa qualité globale. Les pratiques incluent :

- Optimiser la consommation de ressources (CPU, mémoire, réseau) de ton application
- Réduire le poids des pages web et des transferts de données inutiles
- Choisir des hébergeurs utilisant des énergies renouvelables
- Concevoir pour la longévité des appareils (ne pas forcer des mises à jour matérielles inutiles)

Exercice - Jour 2

Évaluation de la responsabilité sociale de ton produit

1. Identifie les 3 impacts sociaux potentiels les plus significatifs de ton produit logiciel (positifs et négatifs). Comment ta démarche qualité prend-elle en compte ces impacts ?
2. Analyse ton produit pour détecter des dark patterns potentiels (consentement obscurcissant, abonnements difficiles à annuler, notifications manipulatrices, etc.). Y en a-t-il ? Comment les éliminer ?
3. Quels groupes d'utilisateurs sont actuellement sous-représentés dans ta stratégie de test ? Propose un plan pour élargir la couverture de test à des profils plus diversifiés.

4. Évalue grossièrement l'empreinte énergétique de ton produit. Quelles sont les 3 optimisations les plus accessibles pour réduire cette empreinte sans compromettre la qualité fonctionnelle ?

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM | TTLST~

Jour 3 - Semaine 8 : Innovation et qualité - comment innover sans sacrifier la fiabilité du produit

La tension perçue entre innovation et qualité est l'une des plus persistantes dans les organisations logicielles. Les équipes qui ont résolu cette tension ont compris qu'innovation et qualité ne sont pas opposées - elles se nourrissent mutuellement quand le cadre organisationnel est bien conçu.

Concepts clés

La fausse dichotomie vitesse-qualité

L'argument classique est : « Pour innover vite, on doit accepter de sacrifier la qualité. » Des décennies de recherche en ingénierie logicielle, notamment les études DORA (DevOps Research and Assessment), ont démontré le contraire. Les équipes les plus performantes (elite performers) sont simultanément les plus rapides *et* les plus fiables. La qualité n'est pas un frein à la vitesse - la dette technique accumulée par des raccourcis de qualité est le frein réel.

Le concept de dette technique comme ennemi de l'innovation

La dette technique est le coût futur des décisions de conception ou d'implémentation prises délibérément ou involontairement sous-optimales. Ses impacts sur la capacité d'innovation :

- Chaque fonctionnalité nouvelle prend de plus en plus longtemps à

implémenter (vélocité décroissante)

- Les changements entraînent des régressions inattendues (instabilité croissante)
- Les nouveaux membres de l'équipe mettent plus de temps à devenir productifs (onboarding difficile)
- L'expérimentation devient risquée car les effets secondaires sont imprévisibles

Gérer la dette technique n'est pas une activité de maintenance - c'est un investissement dans la capacité d'innovation future.

Les pratiques qui permettent d'innover avec qualité

Pratique

Comment elle permet l'innovation

Comment elle maintient la qualité

CI/CD mature

Déploiement fréquent et confiant

Validation automatique à chaque changement

TDD (Test-Driven Development)

Conception emergente guidée par les tests

Couverture de test intégrée dès le départ

Architecture modulaire

Composantes indépendantes, changements localisés

Isolation des défaillances et des changements

Feature flags

Expérimentation en production sans risque

Rollback instantané si problème détecté

Spike exploratoire

Exploration technique rapide et délibérée

Apprentissage avant engagement, pas de dette cachée

Revue d'architecture

Validation des nouvelles directions techniques

Détection précoce des problèmes de conception

Le concept d'error budget comme levier d'innovation

L'error budget (défini à la Semaine 7, Jour 1) est aussi un outil de pilotage de l'innovation. Le principe SRE : si ton error budget est sain (tu n'as pas consommé toute ta marge d'indisponibilité autorisée), tu as la licence d'innover rapidement. Si ton error budget est épuisé, tu priorises la fiabilité et la correction des problèmes avant de reprendre l'innovation.

C'est un cadre objectif qui désidéologise le débat vitesse-qualité : les données décident, pas les opinions.

Les spikes et la gestion de l'incertitude technique

Un spike (terme Agile) est une activité d'exploration technique limitée dans le temps, dont le livrable est de la connaissance plutôt que du code de production. Utilisé pour réduire l'incertitude technique avant de s'engager dans une implémentation, le spike est le mécanisme qui permet d'innover sans créer de dette cachée.

Exercice - Jour 3

Cartographier la tension innovation-qualité dans ton contexte

1. Évalue la dette technique actuelle de ton produit sur une échelle de 1 à 5 (1 = quasi nulle, 5 = bloquante). Quels sont les 3 éléments qui contribuent le plus à cette dette ? Quel est leur impact sur la vélocité d'innovation ?
2. Identifie une initiative d'innovation récente ou planifiée dans ton contexte. Comment as-tu (ou pourrais-tu) la structurer pour minimiser l'impact sur la qualité en production ?
3. Si tu avais 20 % du temps de ton équipe pour rembourser de la dette technique ce trimestre, quels seraient tes 3 investissements prioritaires ? Justifie leur impact sur la capacité d'innovation future.
4. Analyse un choix architectural passé dans ton produit qui t'est aujourd'hui un frein à l'innovation. Qu'aurait-il fallu faire différemment ? Que retiens-tu pour les décisions futures ?

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM | TTLST~

Jour 4 - Semaine 8 : Documentation du produit comme actif de qualité - wikis, ADR, runbooks

La documentation est souvent le parent pauvre de la démarche qualité. On la juge longue à écrire, difficile à maintenir, et peu valorisée dans les équipes axées sur la livraison. Pourtant, la documentation est un actif de qualité fondamental : elle cristallise la connaissance, réduit la dépendance aux individus, et permet à chaque membre de l'équipe de comprendre et maintenir le produit à un niveau de qualité constant.

Concepts clés

Les types de documentation de qualité

Type

Contenu

Audience

Outil typique

ADR (Architecture Decision Records)

Décisions architecturales importantes, contexte, options considérées, décision prise, conséquences

Équipe technique, futurs membres

Git (fichiers Markdown dans le repo), Confluence

Runbooks

Procédures opérationnelles : comment répondre à une alerte, effectuer une tâche de maintenance, diagnostiquer un problème courant

Équipe on-call, opérations

Confluence, Notion, PagerDuty, Opsgenie

Wiki technique

Architecture du système, composantes, flux de données, conventions, environnements, accès

Développeurs, testeurs, nouveaux membres

Confluence, Notion, GitHub Wiki, GitLab Wiki

Spécifications fonctionnelles

Comportement attendu du produit, règles métier, cas limites, critères

d'acceptation

Développeurs, testeurs, product owners

Confluence, Jira, Notion

Documentation API

Contrats d'interface, endpoints, schémas, codes d'erreur, exemples

Développeurs internes et externes

Swagger/OpenAPI, Postman, Redoc

Post-mortems

Historique des incidents, causes racines, actions correctives

Toute l'équipe, management

Confluence, Notion, Google Docs

Architecture Decision Records (ADR) - préserver la mémoire architecturale

Un ADR documente une décision architecturale significative. Sa valeur principale est de préserver le *pourquoi* d'une décision, et non seulement le *quoi*. Sans ADR, les décisions architecturales deviennent des mystères à mesure que les membres de l'équipe changent.

Structure d'un ADR :

- **Titre** : « ADR-001 : Adoption de PostgreSQL comme base de données principale »
- **Statut** : Proposé / Accepté / Déprécié / Supplanté par ADR-XXX
- **Contexte** : Le problème que cette décision adresse
- **Options considérées** : Les alternatives évaluées et leurs compromis
- **Décision** : L'option retenue et la justification

- **Conséquences** : Les impacts positifs et négatifs de cette décision

Runbooks - la qualité opérationnelle documentée

Un runbook de qualité permet à n'importe quel membre on-call de répondre efficacement à un incident, même sans expertise spécifique sur le composant affecté. Un bon runbook contient :

- Le déclencheur (quelle alerte ou quel symptôme provoque l'utilisation de ce runbook)
- L'impact potentiel si non traité
- Les étapes de diagnostic dans l'ordre logique
- Les commandes exactes à exécuter, avec leur effet attendu
- Les procédures d'escalade si le runbook ne résout pas le problème
- L'historique des révisions (date, auteur, changement)

La documentation vivante - maintenir sans souffrir

Le problème de la documentation n'est pas de la créer - c'est de la maintenir. Les pratiques de documentation vivante :

- **Documentation as Code** : traiter la documentation comme du code (versionnée, revue, testée)
- **Mise à jour dans la définition de « done »** : une user story n'est pas terminée si la documentation associée n'est pas mise à jour
- **Documentation générée** : utiliser les tests comme documentation (BDD, contrats de test)
- **Revue de documentation régulière** : auditer trimestriellement pour identifier la documentation obsolète

Exercice - Jour 4

Créer et améliorer ta documentation de qualité

1. Identifie les 3 décisions architecturales les plus importantes de ton produit actuel qui ne sont pas documentées. Rédige un ADR complet pour la plus critique.
2. Choisis le scénario d'incident le plus fréquent dans ton contexte et rédige un runbook complet selon la structure présentée. Fais-le relire par un collègue qui ne connaît pas bien ce composant.
3. Évalue l'état actuel de la documentation de ton produit : quel pourcentage de la connaissance critique est documentée ? Qu'est-ce qui est dans la tête des membres de l'équipe seulement ?
4. Comment intégreras-tu la mise à jour de la documentation dans ton processus de développement quotidien ? Propose une modification concrète de ta définition de « done ».

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM | TTLST~

Jour 5 - Semaine 8 : Onboarding qualité - intégrer la qualité dès l'arrivée de nouveaux membres d'équipe

L'onboarding est le moment le plus influent de l'expérience d'un nouveau membre d'équipe. Ce qu'il apprend, observe et vit dans ses premiers jours va former sa compréhension de ce qu'est la « normale » dans ton équipe. Un onboarding qualité bien conçu est l'investissement le plus rentable que tu puisses faire pour pérenniser ta culture qualité.

Concepts clés

La qualité se transmet, ne s'impose pas

La qualité dans une équipe n'est pas le résultat d'une politique écrite ou d'un processus imposé - c'est le résultat d'une transmission culturelle active. Un nouveau membre observe d'abord comment les pratiques de qualité sont vécues au quotidien par ses collègues expérimentés avant de les adopter lui-même.

Les dimensions de l'onboarding qualité :

Dimension

Ce que le nouveau membre apprend

Comment le transmettre

Connaissance du produit

Architecture, flux de données, composantes, historique des décisions

Visite guidée du wiki, lecture des ADR, sessions d'exploration en pair

Pratiques de test

Stratégie de test, outils, conventions, niveaux de couverture attendus

Session de pair-test, revue de la suite existante, exercice d'écriture de test guidé

Processus qualité

Pipeline CI/CD, critères de merge, revue de code, définition de done

Participation observateur à une revue de code, walkthrough du pipeline

Culture de l'incident

Comment répondre aux alertes, comment faire un post-mortem, comment communiquer

Lecture de post-mortems passés, ombre en on-call, simulation d'incident

Valeurs qualité

Ce qui est valorisé (signaler les problèmes, demander de l'aide, tester avant de livrer)

Observation des comportements modèles, conversations explicites sur la culture

Le plan d'onboarding qualité structuré

Un plan d'onboarding qualité se structure sur 90 jours :

- **Semaine 1 - Observation** : Comprendre le produit, son environnement, ses utilisateurs. Lire la documentation, observer des démos, assister à des rituels d'équipe sans participer activement.
- **Semaine 2-3 - Immersion** : Première contribution guidée (bug fix simple, amélioration de test), pair-programming avec un membre expérimenté, première revue de code.
- **Mois 2 - Autonomie croissante** : Contributions indépendantes avec revue, première participation à la rotation on-call (en duo), participation active aux rituels qualité.
- **Mois 3 - Intégration complète** : Responsabilité complète des contributions, participation autonome à la rotation on-call, rédaction de son premier ADR ou runbook.

Les pièges de l'onboarding qualité

- **Surcharger d'information** : trop de documentation à lire sans ancrage pratique mène à une compréhension superficielle et rapidement oubliée
- **Le mythe du « lancé dans le bain »** : confier immédiatement des tâches complexes sans guidance détruit la confiance et transmet de mauvaises pratiques
- **Ignorer l'aspect culturel** : se concentrer uniquement sur les outils et

processus sans expliquer explicitement les valeurs qualité laisse le nouveau membre deviner

- **L'onboarding comme événement unique** : l'intégration à la culture qualité est un processus de plusieurs mois, pas un événement de 2 jours

Exercice - Jour 5

Concevoir le programme d'onboarding qualité de ton équipe

1. Si un nouveau membre rejoignait ton équipe demain, quelles pratiques de qualité observerait-il naturellement dans les deux premières semaines ? Lesquelles ne verrait-il pas du tout car elles ne sont pas visibles ?
2. Rédige le plan d'onboarding qualité pour ton équipe sur 90 jours. Pour chaque phase, indique les activités, les apprentissages attendus et les indicateurs de réussite.
3. Identifie les 3 lacunes les plus critiques de ton onboarding actuel en matière de qualité. Quelle est la conséquence concrète de chacune sur la qualité du produit ?
4. Comment mesurerais-tu l'efficacité de ton programme d'onboarding qualité ? Quels seraient les indicateurs de succès à 30 jours, 60 jours et 90 jours ?

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM | TTLST~

Jour 6 - Semaine 8 : Mesurer la maturité qualité du produit logiciel - modèles de maturité (CMMI, TMMi)

Mesurer la maturité de ta démarche qualité, c'est comprendre où tu en es sur le chemin de l'excellence, pour savoir où concentrer tes efforts d'amélioration. Les modèles de maturité sont des référentiels structurés qui permettent cette évaluation de façon rigoureuse et comparative.

Concepts clés

CMMI - Capability Maturity Model Integration

Le CMMI (Capability Maturity Model Integration), développé par le Software Engineering Institute (SEI) de Carnegie Mellon University, est le modèle de maturité le plus établi en génie logiciel. Il évalue la maturité des processus d'une organisation selon cinq niveaux :

Niveau CMMI

Nom

Description

Caractéristiques typiques

Niveau 1

Initial

Processus imprévisibles, peu contrôlés, réactifs

Succès dépendant des individus, pas de processus définis, héroïsme récurrent

Niveau 2

Géré

Processus planifiés, exécutés, mesurés et contrôlés

Gestion de projet basique, exigences documentées, revue de produit formelle

Niveau 3

Défini

Processus caractérisés pour l'organisation, proactifs

Processus standardisés et documentés, formation des équipes, métriques collectées

Niveau 4

Quantitativement géré

Processus mesurés et contrôlés de façon quantitative

Objectifs mesurables de performance, analyse statistique des processus

Niveau 5

En optimisation

Amélioration continue des processus basée sur la compréhension quantitative

Prévention des défauts proactive, gestion du changement technologique systématique

TMMi - Test Maturity Model Integration

Le TMMi (Test Maturity Model Integration) est un modèle de maturité spécifiquement dédié aux processus de test logiciel. Développé par la TMMi Foundation, il suit une structure similaire à CMMI avec 5 niveaux :

Niveau TMMi

Nom

Pratiques clés de test

Niveau 1

Initial

Tests ad hoc, aucun processus défini, débogage réactif

Niveau 2

Définition de phase

Tests planifiés après développement, stratégie de test basique, critères d'entrée/sortie

Niveau 3

Intégration

Tests intégrés dans le cycle de vie, organisation de test formelle, tests non fonctionnels

Niveau 4

Gestion et mesure

Métriques de test avancées, contrôle statistique, revues par pairs systématiques

Niveau 5

Optimisation

Prévention des défauts, amélioration continue du processus de test, gestion de la qualité totale

Les métriques de maturité qualité pratiques

Au-delà des modèles formels, tu peux évaluer la maturité qualité de ton équipe avec des métriques pratiques :

- **Métriques DORA** : Deployment Frequency, Lead Time for Changes, Change Failure Rate, Mean Time To Recovery
- **Couverture de test** : pourcentage de code couvert par des tests automatisés, par niveau (unitaire, intégration, E2E)
- **Taux de défauts en production** : nombre de bugs découverts en

production par sprint ou par release

- **Taux de complétion des actions correctives** : pourcentage d'actions correctives de post-mortem complétées à temps
- **Vélocité de remboursement de la dette technique** : proportion du temps investie dans la réduction de la dette technique

Exercice - Jour 6

Évaluer et planifier la progression en maturité qualité

1. À quel niveau CMMI positionnes-tu honnêtement ton organisation aujourd'hui ? Justifie en identifiant 3 comportements observables qui supportent cette évaluation.
2. À quel niveau TMMi positionnes-tu ton processus de test actuel ? Quelles pratiques du niveau suivant pourrais-tu commencer à implémenter dès maintenant ?
3. Mesure les 4 métriques DORA de ton produit pour les 90 derniers jours. Compare avec les benchmarks de l'industrie (elite performers : déploiements multiples par jour, lead time inférieur à une heure, taux d'échec inférieur à 5 %, MTTR inférieur à une heure). Où te situes-tu ?
4. Conçois un plan de progression de maturité qualité sur 12 mois : quels sont les 3 domaines prioritaires d'amélioration et quelle serait la prochaine action concrète pour chacun ?

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM

Jour 7 - Semaine 8 : Ton manifeste personnel de qualité pour le produit logiciel

Tu arrives au dernier jour de la Semaine 8, et donc à la veille de la Grande Totalisation de L2. Ce Jour 7 est différent des précédents : il ne t'apporte pas de nouveaux concepts à assimiler. Il t'invite à synthétiser, articuler et formaliser ta vision personnelle de la qualité logicielle - ce que l'on appelle un manifeste personnel de qualité.

Qu'est-ce qu'un manifeste de qualité ?

Un manifeste de qualité est une déclaration personnelle et professionnelle de tes valeurs, principes et engagements en matière de qualité logicielle. À l'image du Manifeste Agile (2001), qui a transformé l'industrie en 68 mots, ton manifeste n'a pas besoin d'être long pour être puissant. Il doit être *vrai*, *personnel* et *actionnable*.

Les composantes d'un manifeste de qualité efficace

Composante

Question directrice

Exemple

Définition personnelle de la qualité

Pour moi, la qualité d'un produit logiciel signifie...

« La qualité, c'est qu'un produit réponde avec précision aux besoins réels de ses utilisateurs, de façon fiable, accessible et éthique, maintenant et dans le futur. »

Valeurs fondamentales

Quels principes ne suis-je pas prêt à compromettre ?

« Je valorise la transparence sur la perfection apparente, l'apprentissage continu sur la stabilité confortable, et la responsabilité collective sur le blâme individuel. »

Engagements concrets

Quelles pratiques vais-je incarner quotidiennement ?

« Je m'engage à tester avant de livrer, à documenter mes décisions, à partager mes apprentissages des défaillances et à défendre la qualité même sous pression. »

Vision d'impact

Quel monde meilleur crée un produit logiciel de qualité ?

« Je crois qu'un produit logiciel de qualité améliore concrètement la vie des personnes qui l'utilisent et que cette amélioration justifie chaque investissement en qualité. »

Pourquoi formaliser ton manifeste ?

Mettre par écrit tes valeurs et engagements de qualité a des effets concrets :

- Il t'ancre dans tes convictions quand la pression de livraison pousse aux compromis
- Il te sert de boussole pour les décisions difficiles où la bonne réponse n'est pas évidente
- Il te permet de communiquer clairement ta vision à tes collègues, gestionnaires et parties prenantes
- Il évolue avec toi - le revisiter dans 6 mois ou 2 ans te donnera une mesure précise de ta croissance professionnelle

Exercice - Jour 7 : Rédiger ton manifeste personnel de qualité

Prends le temps qu'il faut. Cet exercice n'a pas de bonne ou mauvaise

réponse. Il n'y a que ta réponse, honnête et réfléchie.

1. **Ta définition de la qualité** : En 3 à 5 phrases, rédige ta définition personnelle de la qualité d'un produit logiciel. Elle doit refléter ce que tu as appris sur les 8 semaines, mais surtout ce que tu crois profondément.
2. **Tes 5 valeurs fondamentales** : Liste les 5 valeurs qui guident ta pratique de la qualité. Pour chacune, donne un exemple concret de comment tu les vis (ou veux les vivre) dans ton travail quotidien.
3. **Tes 5 engagements pratiques** : Quelles sont les 5 pratiques de qualité que tu t'engages à incarner dans ton travail, même quand c'est difficile ? Formule-les comme des engagements à la première personne : « Je m'engage à... »
4. **Ta vision d'impact** : Pourquoi la qualité logicielle est-elle importante pour toi, au-delà de ta carrière personnelle ? Quel est l'impact que tu veux avoir sur les utilisateurs, sur ton équipe, sur l'industrie ?
5. **Ton manifeste complet** : Compile les réponses précédentes en un texte cohérent d'une page maximum. C'est ton manifeste personnel de qualité - date-le et garde-le.

Questions de réflexion - Clôture Semaine 8

- En quoi ta vision de la qualité logicielle a-t-elle évolué depuis le début de L2 ? Qu'est-ce qui a changé dans ta compréhension ?
- Quelle dimension de la Semaine 8 (éthique, responsabilité sociale, innovation, documentation, onboarding, maturité) a eu le plus grand impact sur ta façon de penser la qualité ? Pourquoi ?
- Comment vas-tu partager ton manifeste avec ton équipe ? Penses-tu qu'un exercice similaire collectif pourrait avoir un impact sur la culture qualité de

ton organisation ?

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM | ŢŢĻŞŢ~

ŢŢĻŞŢ~ - Totalisation - Semaine 8

Vision holistique du produit logiciel et excellence durable

Les deux dimensions de la Totalisation

Dimension 1 - Substantif linguistique

"Action d'additionner ; action de considérer l'ensemble, le total, de synthétiser."

Dimension 2 - Philosophie ORIGINALIUM

"Rassembler et réaliser totalement dans son ensemble - avant, pendant et après - en continu, sans interruption."

Exercice de Totalisation - Semaine 8

Totaliser la Semaine 8, c'est intégrer les dimensions humaines, éthiques et organisationnelles de la qualité dans une vision holistique cohérente et personnelle.

Tu vas maintenant synthétiser comment chaque dimension contribue à l'excellence durable.

Synthèse des 7 dimensions de la Semaine 8

Dimension	Contribution à l'excellence durable	Interconnexion
Éthique (Jour 1)	La qualité qui respecte et protège ses utilisateurs est la seule qualité durable	Fondement de la confiance des utilisateurs
Responsabilité sociale (Jour 2)	Un produit conscient de son impact sociétal se construit pour durer	Ancrage de la qualité dans le contexte humain réel
Innovation (Jour 3)	L'excellence durable exige la capacité d'évoluer sans se dégrader	Équilibre dynamique entre stabilité et mouvement
Documentation (Jour 4)	La connaissance documentée survit aux individus et perpétue la qualité	Mémoire organisationnelle de l'excellence
Onboarding (Jour 5)	La qualité transmise dès le départ se multiplie avec chaque nouveau membre	Vecteur de propagation de la culture qualité
Maturité (Jour 6)	Mesurer où tu en es est la condition pour progresser de façon consciente	Miroir objectif de l'excellence organisationnelle
Manifeste (Jour 7)	Une vision personnelle claire ancre l'engagement dans la durée	Boussole individuelle au service de l'excellence collective

Exercice de Totalisation - Semaine 8

Totalise ta vision holistique et éthique du produit logiciel en

répondant à ces questions intégratrices :

1. **La dimension technique** : Comment les pratiques techniques de qualité (tests, observabilité, CI/CD, architecture) que tu as étudiées tout au long de L2 contribuent-elles à l'excellence durable ? En quoi sont-elles insuffisantes si elles ne sont pas ancrées dans des valeurs éthiques ?

2. **La dimension humaine** : Comment les personnes - les utilisateurs, les membres de l'équipe, les nouveaux arrivants - sont-elles au coeur de la qualité durable ? Que se passe-t-il quand on ignore cette dimension ?

3. **La dimension organisationnelle** : Comment la culture, la maturité des processus, la documentation et l'apprentissage continu créent-ils les conditions de l'excellence durable ? Que manque-t-il sans cette dimension ?

4. **La synthèse holistique** : Rédige un paragraphe qui totalise comment les trois dimensions - technique, humaine et organisationnelle - s'articulent pour créer un produit logiciel d'excellence durable. Ce paragraphe doit être ta vision propre, nourrie de tout ce que tu as appris sur les 8 semaines.

5. **L'engagement concret** : Identifie les 3 changements les plus importants que tu vas apporter à ta pratique professionnelle à la suite de la Semaine 8. Pour chacun : qu'est-ce que tu vas faire différemment, dès la semaine prochaine ?

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM | TTLST~

ṪṪḶṢṬ~ - Grande Totalisation

L2 - Réconciliation avec le produit logiciel - Semaines 1 à 8

La Grande Totalisation - Philosophie ORIGINALIUM

"Rassembler et réaliser totalement dans son ensemble - avant, pendant et après - en continu, sans interruption. C'est le moment de considérer l'ensemble de L2 comme une totalité cohérente - non pas une série de semaines, mais un voyage complet vers la réconciliation avec le produit logiciel."

1. Rappel des apprentissages clés - Semaines 1 à 8

Ce rappel couvre les semaines 1 à 6 de L2 (Parties A et B) et les semaines 7-8 (Partie C) pour une synthèse complète des 8 semaines.

Semaine 1 - Fondements du produit logiciel

Définir ce qu'est un produit logiciel, le distinguer d'un projet, comprendre la notion de valeur délivrée aux utilisateurs. La qualité comme propriété intrinsèque du produit, non comme activité séparée. Les modèles de qualité logicielle (ISO 25010, FURPS+) comme cadres de référence structurés.

Semaine 2 - Exigences et qualité dès la source

La qualité commence dans la compréhension des besoins. Exigences fonctionnelles et non fonctionnelles, critères d'acceptation, définition de done. La gestion des exigences changeantes comme facteur de qualité. Le coût de corriger un défaut croît exponentiellement avec le temps - la qualité en amont est la plus rentable.

Semaine 3 - Architecture et qualité structurelle

L'architecture logicielle comme décision de qualité fondamentale. Couplage, cohésion, principes SOLID, dette technique. Les caractéristiques architecturales (scalabilité, maintenabilité, testabilité) comme attributs de qualité. Les patterns architecturaux et leurs compromis qualité.

Semaine 4 - Stratégie de test et pyramide

La stratégie de test comme plan intégré de vérification de la qualité. La pyramide de test (unitaires, intégration, E2E) et l'équilibre entre vitesse et confiance. Les tests automatisés comme filet de sécurité de la qualité. La couverture de code comme indicateur partiel de la confiance dans les tests.

Semaine 5 - Intégration continue et qualité dans le pipeline

CI/CD comme infrastructure de qualité continue. Le pipeline comme suite d'étapes de validation automatique. Les quality gates comme barrières objectives qui empêchent les défauts de progresser. La rapidité de feedback

comme facteur clé d'efficacité de la démarche qualité.

Semaine 6 - Qualité non fonctionnelle et performance

Les attributs de qualité au-delà du fonctionnel : performance, sécurité, accessibilité, fiabilité, maintenabilité. Les tests de charge, de stress, de résilience. La performance comme dimension de qualité perçue par l'utilisateur. L'intégration des tests non fonctionnels dans le pipeline CI/CD.

Semaine 7 - Qualité en production et feedback loop

La production comme environnement de vérité ultime de la qualité. Monitoring, alertes, SLO, SLA. Gestion des incidents avec méthode et transparence. L'observabilité (logs, métriques, traces) comme infrastructure cognitive de compréhension. Post-mortem blameless comme moteur d'apprentissage. La culture qualité comme substrat durable de toutes les pratiques.

Semaine 8 - Vision holistique et excellence durable

L'éthique (accessibilité, confidentialité, sécurité) comme pilier de qualité, non comme option. La responsabilité sociale et les biais dans les produits logiciels. Innovation et qualité comme complémentaires. La documentation comme actif de qualité vivant. L'onboarding comme vecteur de transmission de la culture qualité. Les modèles de maturité (CMMI, TMMi) comme miroirs objectifs de la progression. Le manifeste personnel comme boussole durable.

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM | TTLST~

2. Exercice de synthèse - Ta carte mentale du produit logiciel de qualité

Une carte mentale (mind map) est une représentation visuelle et structurée de la façon dont les concepts s'interconnectent dans ta compréhension. Construire la carte mentale du produit logiciel de qualité, c'est totaliser L2 dans une structure que tu pourras réutiliser, partager et enrichir tout au long de ta carrière.

Exercice 2 - Construire ta carte mentale complète

Instructions :

1. **Centre de la carte** : « Produit logiciel de qualité » - c'est ton concept central.
2. **Branches principales** (une par semaine de L2) : pour chacune des 8 semaines, crée une branche principale avec le thème de la semaine.
3. **Sous-branches** : Pour chaque semaine, identifie 3 à 5 concepts clés et place-les en sous-branches.
4. **Connexions transversales** : Identifie et dessine au moins 5 connexions entre des concepts de semaines différentes (ex. : la couverture de test de la Semaine 4 connectée aux quality gates de la Semaine 5).
5. **Couleurs et visuels** : Utilise des couleurs différentes pour chaque semaine. Ajoute des symboles visuels pour marquer les concepts qui t'ont le plus marqué.

6. **Révision** : Une fois ta carte tracée, identifie les 3 connexions les plus importantes que tu n'avais pas anticipées. Que révèlent-elles sur la cohérence de ta vision ?

Tu peux réaliser cet exercice sur papier, dans un outil de mind mapping numérique (Miro, Mural, Lucidchart, XMind) ou dans n'importe quel format qui te convient. L'important est le processus de connexion, pas l'outil.

Guide de connexions transversales à explorer

Connexion	Concept 1	Concept 2	Nature du lien
1	Exigences (Semaine 2)	Critères d'acceptation des tests (Semaine 4)	Les exigences définissent ce que les tests vérifient
2	Architecture testable (Semaine 3)	Pyramide de test (Semaine 4)	L'architecture module la facilité de test à chaque niveau
3	Pipeline CI/CD (Semaine 5)	Canary releases (Semaine 7)	Le pipeline est le mécanisme de déploiement progressif
4	SLO et error budget (Semaine 7)	Innovation et qualité (Semaine 8)	L'error budget est la mesure objective de la licence d'innover
5	Post-mortem blameless (Semaine 7)	Culture qualité (Semaine 7 et 8)	Le post-mortem est à la fois pratique et rituel culturel
6	Modèle ISO 25010 (Semaine 1)	Éthique et accessibilité (Semaine 8)	Les dimensions éthiques complètent et enrichissent les modèles formels
7	Observabilité (Semaine 7)	Performance non fonctionnelle (Semaine 6)	L'observabilité mesure en production ce que

Connexion	Concept 1	Concept 2	Nature du lien
			les tests de performance anticipent

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM | TTLST~

3. Exercice de consolidation - Tes forces et tes axes d'amélioration

La consolidation de tes apprentissages passe nécessairement par une évaluation honnête et courageuse de ta situation actuelle. Identifier tes forces te permet de les amplifier et de t'en servir comme leviers. Identifier tes axes d'amélioration te donne une direction claire pour la suite de ton parcours de développement professionnel.

Exercice 3 - Tes 3 forces et tes 3 axes d'amélioration

Partie A - Tes 3 forces en qualité logicielle

Pour chaque force, réponds aux trois sous-questions :

1. Quelle est cette force ? (ex. : expertise en test automatisé, culture du post-mortem, solide compréhension de l'architecture testable)
2. Comment se manifeste-t-elle concrètement dans ton travail quotidien ? Donne un exemple récent.
3. Comment peux-tu amplifier cette force pour avoir un impact encore plus grand sur la qualité de ton produit et sur ta culture d'équipe ?

Partie B - Tes 3 axes d'amélioration prioritaires

Pour chaque axe d'amélioration, réponds aux trois sous-questions :

1. Quel est cet axe d'amélioration ? (ex. : observabilité insuffisante du produit en production, processus de gestion des incidents peu structuré, documentation technique quasi absente)
2. Quel est l'impact concret de cette lacune sur la qualité de ton produit aujourd'hui ? Sois précis et objectif.
3. Quelle est la première action concrète que tu pourrais prendre dès la semaine prochaine pour commencer à combler cette lacune ? Elle doit être réaliste, spécifique et mesurable.

Grille d'auto-évaluation - 8 dimensions de L2

Dimension	Force (1-5)	Priorité d'amélioration (1-5)	Notes
Compréhension du produit logiciel et de sa valeur	___	___	
Gestion des exigences et critères de qualité	___	___	
Architecture logicielle et qualité structurelle	___	___	
Stratégie de test et pyramide de test	___	___	
Intégration continue et pipeline qualité	___	___	
Qualité non fonctionnelle et performance	___	___	
Qualité en production et	___	___	

Dimension	Force (1-5)	Priorité d'amélioration (1-5)	Notes
feedback loop			
Vision holistique, éthique et culture qualité	___	___	

Force : 1 = débutant, 5 = expert. Priorité d'amélioration : 1 = non prioritaire, 5 = très urgent.

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM | TTLST~

4. Exercice de projection - Appliquer dans ton contexte professionnel réel

La connaissance qui ne se traduit pas en action reste de la théorie. Cet exercice de projection est celui qui convertit tes apprentissages de L2 en plan d'action concret pour ton environnement professionnel réel. C'est ici que L2 cesse d'être un livre de formation et devient un levier de transformation.

Exercice 4 - Plan d'application dans ton contexte réel

Étape 1 - Cartographie de ton contexte

Décris ton contexte professionnel actuel selon les axes suivants :

- **Ton produit logiciel** : type, utilisateurs, domaine métier, criticité
- **Ta position** : rôle, périmètre de responsabilité, niveau d'influence sur les décisions qualité
- **Ton équipe** : taille, maturité actuelle, culture qualité perçue

- **Tes contraintes** : temps, budget, appui managérial, dette technique existante

Étape 2 - Les 5 initiatives qualité prioritaires

Basé sur tes apprentissages de L2 et sur ton contexte, identifie les 5 initiatives qualité qui auraient le plus grand impact dans ton contexte dans les 6 prochains mois. Pour chacune :

#

Initiative

Semaine L2 associée

Impact attendu

Délai réaliste

Sponsor / Allié

1

2

3

4

5

Étape 3 - L'initiative 90 jours

Choisis l'initiative la plus prioritaire de ton tableau. Rédige un plan d'implémentation sur 90 jours :

- **Mois 1** : Analyse, préparation, obtenir les appuis nécessaires, définir les métriques de succès
- **Mois 2** : Implémentation pilote avec un sous-ensemble de l'équipe ou du produit
- **Mois 3** : Extension, mesure des résultats, ajustements, préparation du

déploiement complet

Étape 4 - Les obstacles anticipés et tes stratégies

Identifie les 3 obstacles les plus probables à l'application de tes apprentissages de L2 dans ton contexte. Pour chacun, quelle stratégie vas-tu employer pour le surmonter ?

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM | TTLST~

5. Totalisation finale de L2

TTLST~ - Totalisation Finale

Dimension 1 - Substantif linguistique :

"Action d'additionner ; action de considérer l'ensemble, le total, de synthétiser."

Dimension 2 - Philosophie ORIGINALIUM :

"Rassembler et réaliser totalement dans son ensemble - avant, pendant et après - en continu, sans interruption."

L'exercice ultime - Ta définition totalisée du produit logiciel de qualité

Tu as traversé 8 semaines d'exploration rigoureuse du produit logiciel sous toutes ses facettes. Tu as totalisé chaque semaine individuellement. Tu as construit ta carte mentale, évalué tes forces, projeté tes actions. Il est maintenant temps de

réaliser la totalisation la plus importante : écrire ta propre définition du produit logiciel de qualité.

Cette définition doit :

- Être **complète** : couvrir les dimensions techniques, humaines, éthiques et organisationnelles
- Être **personnelle** : refléter ta vision propre, nourrie de tes expériences et de tes apprentissages
- Être **actionnable** : permettre à quelqu'un qui la lit de comprendre ce que tu vas faire différemment
- Être **inspirante** : porter une vision de l'excellence qui motive, pas seulement une liste de critères
- Être **durable** : rester vraie dans 5 ans, même si les outils et les technologies ont changé

Exercice ultime - Rédige ta définition totalisée

Instructions :

1. Prends au moins 30 minutes pour cet exercice. Éloigne les distractions.
2. Relis ton manifeste personnel de qualité (Jour 7, Semaine 8).
3. Relis les totalisations de chaque semaine (1 à 8).
4. Rédige ta définition totalisée du produit logiciel de qualité en 150 à 300 mots. Elle peut prendre la forme d'un texte continu, d'une liste de principes, ou d'une combinaison des deux.
5. Date et signe ta définition. C'est un document vivant - tu peux la réviser à mesure que ton expérience évolue, mais la version du jour de ta Grande Totalisation L2 est un jalon important.

Espace pour ta définition totalisée :

[Rédige ici ta définition totalisée du produit logiciel de qualité]

Signature et date : _____

Félicitations - Tu as complété L2 - Réconciliation avec le produit logiciel

Tu viens d'accomplir quelque chose que peu de professionnels prennent le temps de faire : comprendre en profondeur, de façon structurée et personnelle, ce qu'est un produit logiciel de qualité dans toutes ses dimensions. Cette compréhension est maintenant en toi. Elle va nourrir chaque décision, chaque conversation et chaque contribution que tu apporteras à tes équipes et à tes produits dans les années qui viennent.

La prochaine étape t'attend. **TTLST~**

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM | TTLST~

Pont vers L3

Réconciliation avec le cycle de vie de la réalisation du logiciel (SDLC)

De L2 à L3 - Tu comprends le produit. Maintenant, comprends le voyage.

Tu viens de totaliser L2 dans sa complétude. Tu comprends maintenant le produit logiciel de qualité dans toute sa profondeur - ses dimensions techniques, humaines, éthiques, opérationnelles et organisationnelles. Tu sais ce qu'est un produit de qualité, comment il se teste, comment il se mesure, comment il se surveille en production, et comment il se construit dans une culture d'excellence.

Mais un produit logiciel ne s'invente pas - il se réalise. Il suit un voyage structuré, depuis l'idée initiale jusqu'à sa mise entre les mains des utilisateurs. Ce voyage a un nom : le cycle de vie de réalisation du logiciel, ou SDLC (Software Development Life Cycle). C'est précisément ce que L3 va t'apprendre à maîtriser.

L3 te guidera à travers toutes les phases de ce cycle : l'idéation et la planification, l'analyse des besoins, la conception, le développement, les tests formels, le déploiement et la maintenance. Tu exploreras les grandes méthodologies - de Waterfall à Agile, de Scrum à Kanban, de SAFe à DevOps - et tu comprendras comment chacune structure le voyage de réalisation du logiciel pour maximiser la qualité à chaque étape.

La connexion est directe et profonde : tout ce que tu as appris dans L2 sur la qualité du produit trouvera sa place dans le cycle de vie que tu vas découvrir dans L3. Tu ne seras pas un apprenant passif de nouvelles notions - tu seras un praticien de la qualité qui comprend le cycle de vie avec les yeux d'un expert du produit.

L3 t'attend. Le voyage continue. ŢŢĻŞŢ~

Ce qui t'attend dans L3

- Les phases du SDLC et leur connexion à la qualité du produit
- Les méthodologies agiles et leur impact sur la démarche qualité

- La place du testeur et du praticien qualité dans chaque phase du cycle
- Les pratiques de qualité intégrées dans le processus de réalisation
- La gouvernance de la qualité à travers le cycle de vie complet
- La réconciliation finale entre le produit que tu comprends et le processus qui le crée

---

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM | TTLST~

Notes personnelles

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM | TTLST~

Notes personnelles

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM | TTLST~

Notes personnelles

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM

| T̰T̰L̰S̰T~

T̰T̰L̰S̰T~

BEASTEST ORIGINALIUM

Méthode d'excellence en qualité logicielle

L2 - Réconciliation avec le produit logiciel

Partie C - Semaines 7-8 et Grande Totalisation

© Léo Michel Bélanger - Tous droits réservés

Images et couverture par Soléane Bélanger

Publié par ORIGINALIUM

Longueuil, QC, Canada - 2026

La prochaine étape :

L3 - Réconciliation avec le cycle de vie de la réalisation du logiciel (SDLC)

TTLST~

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM | TTLST~